
A5 Advanced SQL Topics (2026)

Relational Databases
RDB (MTD.ba VZ)
SS26 2026

Submission via Moodle

16 Points

The following assignments all result in SQL queries for the Steam games database. In your final submission, please provide a .sql file with all the assignments you completed, in the following format:

```
1  -- 01:
2  SELECT * FROM games;
```

Please make sure to have a **comment** with the **assignment number** and below that the SQL statement(s) for the respective assignment.

For each assignment please also prepare at least two test cases, one succeeding and one failing (except exercises 3,4,5 and 7).

Exercise 1 (2 points) Write a transaction that adds a new game to the database and links it to two genres (**Action** and **Strategy**) and a developer (**Bethesda Softworks**). Ensure that the developer and genres exist before adding. Rollback if any step fails. Use a dedicated test steam_appid that does not collide with existing rows.

Exercise 2 (2 points) Write a transaction to delete a game, and ensure that all related genre, developer, platform, and category associations are removed. If the game does not exist or any step fails, rollback. (The reference deletes the test game from Exercise 1, steam_appid = 999602, and uses a non-existent ID for a failure path.)

Exercise 3 (2 points) Add a fulltext index on the 'name' and 'review_score_desc' columns of the 'games' table. Then, write a query that finds games containing both keywords "Overwhelmingly" and "Positive" using 'MATCH ... AGAINST'.

Exercise 4 (1 points) Compare the performance of a query that searches for a game by name using 'LIKE '%RPG%'' versus using a fulltext index (you can use the fulltext index from the previous exercise).

Exercise 5 (3 points) Create a stored function is_underrated(appid INT) that returns 1 if a game has at least **80%** positive user reviews but a Metacritic score of **55** or lower. It should return 0 otherwise. Make sure to return 0 as well if the game has no user review.

Exercise 6 (2 points) Write a stored procedure 'add_genre_to_game(appid INT, genreName VARCHAR(255))' that checks if the genre exists, and if so, links it to the game. If not, throw an error using 'SIGNAL'.

Exercise 7 (2 points) Write a trigger that logs the deletion of a game (name, date, reason) into a 'game_deletion_log' table.

Exercise 8 (2 points) Write a 'BEFORE INSERT' trigger that ensures games have a 'price_initial' value of 0 if 'is_free = 1', otherwise throws an error.